

sheLLaMa Admin Manual

Deployment, Configuration & Operations

Version 1.0 — May 2026

Contents

1	Architecture Overview	2
2	System Requirements	2
2.1	Backend Server	2
2.2	Frontend Server	2
3	Deployment	2
3.1	Quick Start (Single Server)	2
3.2	Distributed Deployment (Ansible)	2
3.3	Manual Deployment	3
3.4	backends.json	3
4	Admin Console	3
5	Command Permissions	3
5.1	Managing Permissions	4
6	Authentication	4
6.1	API Keys	4
6.2	SSO (OIDC)	4
6.3	Roles	4
7	Cloud Fallback	5
7.1	OpenRouter (Cloud)	5
7.2	LiteLLM (Self-Hosted)	5
7.3	Auto-Fallback Mode	5
8	Load Balancing	5
9	Rate Limiting & Budgets	6
10	Prompt Caching	6
11	OpenAI-Compatible API	6
12	TLS & mTLS	6
12.1	Certificate Management	6
12.2	Environment Variables	7
13	Webhooks	7

14 Cloud Cost Estimation	7
14.1 Costs Page (/costs)	7
14.2 Cloud Usage Page (/cloud-usage)	7
14.3 Enterprise Comparison	8
15 Monitoring & Operations	8
15.1 Service Management (Linux)	8
15.2 Health Checks	8
15.3 Reset Counters	8
15.4 Task Timeouts	8
16 Environment Variables (Complete)	8
17 Troubleshooting	9

1 Architecture Overview

Clients --> Frontend (:5000) --> Backend Farm

```
|-- Backend 1 (:5000)
|-- Backend 2 (:5000)
|-- Backend N (:5000)
```

External tools --> /v1/chat/completions (OpenAI-compatible)

- **Backend** (backend/app.py) — Ollama worker, task queue, cloud fallback, image generation subprocess
- **Frontend** (frontend/app-distributed.py) — Load balancer, routing, caching, auth, rate limiting, health checks, webhooks, admin console
- **Clients** — Bash CLI, PowerShell CLI, integrations, GUIs, REST API consumers

2 System Requirements

2.1 Backend Server

Tier	CPU	RAM	Storage	Models
Minimum	8 cores	16GB	50GB	7B (30–60s/response)
Recommended	16 cores	32GB	100GB	13B–14B (1–3min/response)
Large	32+ cores	64GB+	200GB	32B+ (not recommended for CPU)

2.2 Frontend Server

4 cores, 8GB RAM, 20GB storage.

3 Deployment

3.1 Quick Start (Single Server)

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5-coder:7b

# Clone and run
git clone https://github.com/pfworks/sheLLaMa.git
cd shellama
python3 -m venv venv && source venv/bin/activate
pip install flask ollama pyyaml requests psutil
python backend/app.py
```

3.2 Distributed Deployment (Ansible)

```
cp deploy/inventory.ini.example inventory.ini
cp deploy/inventory-frontend.ini.example inventory-frontend.ini
cp deploy/backends.json.example backends.json
# Edit each file with your server details
```

```
ansible-playbook -i inventory.ini deploy/deploy.yml
ansible-playbook -i inventory-frontend.ini deploy/deploy-frontend.yml
```

3.3 Manual Deployment

```
# Frontend
scp frontend/app-distributed.py root@frontend:/usr/local/bin/
scp shared/*.py root@frontend:/usr/local/shared/
scp frontend/web/*.html root@frontend:/export/html/
ssh root@frontend "systemctl restart ansible-ollama-frontend"

# Backend
scp backend/app.py root@backend:/usr/local/bin/
scp shared/*.py root@backend:/usr/local/shared/
ssh root@backend "systemctl restart ansible-ollama"
```

3.4 backends.json

```
{
  "backends": [
    {"url": "http://192.168.1.230:5000", "weight": 10,
      "max_model": "qwen2.5-coder:7b"},
    {"url": "http://192.168.1.233:5000", "weight": 10,
      "max_model": "qwen2.5-coder:14b"}
  ]
}
```

Higher weight = higher priority. Score = queue_size - (weight * 0.1), lowest wins.

4 Admin Console

Access at <http://frontend:5000> (redirects to /status).

Page	URL	Description
Status	/status	Total requests, tokens, active backends, queue, fallback toggle
Backends	/backends	Per-backend: online/offline, CPU/RAM, weight, models, active task
Stats	/stats	Charts: queue size and token usage over time
Costs	/costs	Cloud cost tracking by time range, fallback spend
Cloud Usage	/cloud-usage	Per-IP and per-API-key cost breakdown
Settings	/settings	Model aliases, toggles, webhooks, command permissions

5 Command Permissions

The agentic shell uses three permission tiers for commands:

Tier	Behavior	Examples
------	----------	----------

Allow	Auto-run	ls, cat, grep, git status, curl, ps, echo
Prompt	Asks user	pip install, python3, mkdir, git commit
Block	Always refused	rm -rf /, mkfs, git push -force, fork bombs

5.1 Managing Permissions

Via Settings page: Navigate to `/settings` → Command Permissions.

- **Auto-Allowed Patterns:** editable regex list. Add/remove freely.
- **Blocked — Built-in:** immutable safety rules. Cannot be removed.
- **Blocked — Custom:** admin-added block rules. Can be removed.

Via API:

```
# Get current permissions
curl http://frontend:5000/api/permissions

# Add an allow pattern
curl -X POST http://frontend:5000/api/permissions \
-H "Content-Type: application/json" \
-d '{"add_allowed": "^npm\\s"}'

# Add a custom block pattern
curl -X POST http://frontend:5000/api/permissions \
-H "Content-Type: application/json" \
-d '{"add_blocked": "dangerous_script\\.sh"}'

# Remove a custom block (built-in blocks cannot be removed)
curl -X POST http://frontend:5000/api/permissions \
-H "Content-Type: application/json" \
-d '{"remove_blocked": "dangerous_script\\.sh"}'
```

The CLI fetches permissions from the server on startup (2s timeout, falls back to hardcoded defaults if unreachable).

6 Authentication

Optional—disabled when `/etc/shellama/auth.json` doesn't exist.

6.1 API Keys

```
# Client usage
export SHELLAMA_API_KEY=sk-your-key
# Or header: X-API-Key: sk-your-key
```

6.2 SSO (OIDC)

Supports Keycloak, Azure AD, Authentik. Configure in `auth.json`:

```
cp deploy/auth.json.example /etc/shellama/auth.json
# Edit with your SSO provider details
```

6.3 Roles

Role	API Access	Web UI	Models	Cloud
admin	All endpoints	Full (modify)	All	Yes
user	Chat, gen, explain	View only	Configured	Configurable
viewer	Read-only	View only	None	No

7 Cloud Fallback

When local models produce poor output, requests can automatically retry via cloud.

7.1 OpenRouter (Cloud)

```
# On each backend (systemd override):
Environment="OPENROUTER_API_KEY=sk-or-v1-your-key"
Environment="OPENROUTER_MODEL=anthropic/claude-3.5-sonnet"
Environment="USE_CLOUD_FALLBACK=true"
```

7.2 LiteLLM (Self-Hosted)

```
Environment="OPENROUTER_API_KEY=sk-anything"
Environment="OPENROUTER_MODEL=fallback"
Environment="OPENROUTER_URL=http://litellm-host:4000/v1/chat/completions"
Environment="USE_CLOUD_FALLBACK=true"
```

7.3 Auto-Fallback Mode

Toggle via Status page or API:

```
curl -X POST http://frontend:5000/auto-fallback \
-H "Content-Type: application/json" \
-d '{"enabled": true}'
```

When enabled, poor local responses auto-retry via cloud without client confirmation.

8 Load Balancing

- Weighted routing: $\text{score} = \text{queue_size} - (\text{weight} * 0.1)$, lowest wins
- Model size filtering: requested model must be $\leq$ backend's `max_model`
- Multi-file analysis runs in parallel across backends
- Health checks every 30s; 3 consecutive failures $\rightarrow$ unhealthy (skipped)
- Auto-recovery when backend responds again
- Retry up to 2x on different backends on failure

9 Rate Limiting & Budgets

Per-key configuration in `auth.json`:

```
"api_keys": {
  "sk-team-key": {
    "name": "team",
    "role": "user",
    "rate_limit": {"rpm": 30, "tpd": 500000},
    "budget": {"max_daily": 5.00}
  }
}
```

- rpm: requests per minute
- tpd: tokens per day
- budget.max_daily: max cloud fallback spend per day (USD)
- Budget warning webhook fires at 80%
- Returns HTTP 429 when exceeded

10 Prompt Caching

- Key: SHA256(endpoint + model + content)
- TTL: 5 minutes (configurable via `SHELLAMA_CACHE_TTL`, 0 to disable)
- Max entries: 500
- Skips: conversations, force_cloud, errors
- Stats tracked: cached_requests, tokens_saved

11 OpenAI-Compatible API

Use sheLLaMa as a drop-in replacement for OpenAI:

```
export OPENAI_API_BASE=http://frontend:5000/v1
export OPENAI_API_KEY=sk-your-key

curl -X POST http://frontend:5000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer sk-your-key" \
  -d '{"model": "fast", "messages": [{"role": "user",
    "content": "Hello"}]}'
```

Works with: Cursor, Continue, Open WebUI, LangChain, any OpenAI client. Model aliases supported (fast → llama3.2:1b).

12 TLS & mTLS

12.1 Certificate Management

```

bin/generate-certs.sh init                # Generate CA
bin/generate-certs.sh server backend-1 \
  192.168.1.230,localhost                # Server cert
bin/generate-certs.sh client frontend-mtls # Client cert
bin/generate-certs.sh list                # List all certs
bin/generate-certs.sh revoke backend-1    # Revoke

```

12.2 Environment Variables

Variable	Description
SHELLAMA_TLS_CERT	Server TLS certificate (enables HTTPS)
SHELLAMA_TLS_KEY	Server TLS private key
SHELLAMA_TLS_CA	CA cert for client verification (mTLS)
SHELLAMA_BACKEND_CERT	Client cert for frontend→backend mTLS
SHELLAMA_BACKEND_KEY	Client key for frontend→backend mTLS
SHELLAMA_BACKEND_CA	CA to verify backend server certs

13 Webhooks

Events: `backend_down`, `backend_recovered`, `budget_warning`.

Configure via Settings page or API:

```

curl -X POST http://frontend:5000/api/webhooks \
-H "Content-Type: application/json" \
-d '{"add": "https://hooks.example.com/shellama"}'

```

Same event suppressed for 5 minutes (dedup).

14 Cloud Cost Estimation

sheLLaMa tracks every token and projects costs across 28+ cloud models.

14.1 Costs Page (/costs)

- Time range filtering: day/week/month/year/custom
- Hypothetical costs: what local usage would cost on cloud
- Actual fallback spend: real costs from cloud fallback
- Cached savings: tokens served from cache
- Cheapest/costliest provider highlighted

14.2 Cloud Usage Page (/cloud-usage)

- Per-IP cost breakdown with expandable provider detail
- Per-API-key cost breakdown
- Per-cloud-model usage (fallback only)
- Cost range (cheapest to most expensive) per client

14.3 Enterprise Comparison

```
curl "http://frontend:5000/enterprise-costs?users=10&days=30"
```

Compares seat-based subscriptions (Copilot, ChatGPT Team, Claude Team, etc.) vs equivalent per-token API spend.

15 Monitoring & Operations

15.1 Service Management (Linux)

```
sudo systemctl status ansible-ollama          # Backend
sudo systemctl status ansible-ollama-frontend # Frontend
sudo journalctl -u ansible-ollama -f          # Logs
sudo systemctl restart ansible-ollama
```

15.2 Health Checks

```
curl http://frontend:5000/queue-status # Aggregate status
curl http://backend:5000/queue-status  # Single backend
```

15.3 Reset Counters

```
curl -X POST http://frontend:5000/reset-stats # Requests/tokens
curl -X POST http://frontend:5000/reset-cloud-costs # Cost data
curl -X POST http://frontend:5000/reset-all # Everything
```

15.4 Task Timeouts

- Default: 1800s (30 min) per task (SHELLAMA_TASK_TIMEOUT)
- Stale task reaper checks every 10s
- Tasks without heartbeat for 30s are killed
- Frontend→backend timeout: 3600s (1 hour)

16 Environment Variables (Complete)

Variable	Default	Description
SHELLAMA_API	192.168.1.229:5000	API endpoint (clients)
SHELLAMA_MODEL	auto	Default model
SHELLAMA_API_KEY	(empty)	API key
SHELLAMA_TASK_TIMEOUT	1800	Max task runtime (backend)
SHELLAMA_COMPACT_THRESHOLD	24000	Auto-compact threshold
SHELLAMA_CACHE_TTL	300	Prompt cache TTL (0=off)
SHELLAMA_AUTH_FILE	/etc/shellama/auth.json	Auth config
SHELLAMA_WEBHOOK_URL	(empty)	Webhook URL
SHELLAMA_CERT_DIR	/etc/shellama/pki	PKI directory
USE_CLOUD_FALLBACK	false	Enable cloud fallback

OPENROUTER_API_KEY	(empty)	Cloud fallback key
OPENROUTER_MODEL	claude-3.5-sonnet	Cloud fallback model
OPENROUTER_URL	openrouter.ai/...	Cloud endpoint
AI_IMAGE_MODEL	sdxl-turbo	Image model
SHELLAMA_DOWNLOAD_DIR	(current dir)	Image save dir

17 Troubleshooting

No backends available — Check `backends.json`; `max_model` must match or exceed the requested model.

Timeouts — Frontend→backend timeout is 3600s. If tasks exceed `SHELLAMA_TASK_TIMEOUT`, the reaper kills them.

Slow responses — Use smaller models. Add more backends. Check `htop`.

Image generation slow — Use `sd-turbo` instead of `sdxl-turbo`. First request loads model (~90s), subsequent are faster.

Permission endpoint returns 403 — Cannot remove built-in safety rules. Only custom block rules can be removed.

Cloud fallback not triggering — Verify `USE_CLOUD_FALLBACK=true` on backends. Check `OPENROUTER_API_KEY` is set.